# Electro Pi — QA Automation Engineer Technical Assessment

**Scenario:** a multi-tenant, cloud-based POS and inventory web application built on REST APIs and dynamic UI components.

**Repository:** the UI tests (1.3), the API tests (2.1) and the CI pipeline (3.2) are implemented and runnable. They run against a small mock of the application in `mock-app/`, which Playwright starts automatically. The Postman (2.2) and SQL (3.1) answers are given below.

## Part 1: UI Test Automation & Architecture

### 1.1 Framework Choice

**Playwright with JavaScript.**

- **Auto-waiting.** Playwright waits for elements and retries assertions on its own. This suits dynamic screens such as async tables, spinners and toasts.
- **Isolated browser contexts.** Tenant A and Tenant B can be logged in at the same time, in the same test, with no shared cookies. This is ideal for multi-tenant testing.
- **Built-in API client.** The same tool calls the REST API for test setup, cleanup and API tests.
- **Debugging.** HTML report, Trace Viewer, and screenshots and video on failure.
- **Cross-browser.** Chromium, Firefox and WebKit (Safari).

Cypress runs in a single tab, which makes multi-tenant scenarios harder. Selenium needs much more custom wait code to be as stable.

### 1.2 Framework Design

**Page Object Model** with shared components and Playwright fixtures.

```
tests/ui/        UI tests
tests/api/       API tests
pages/           Page Objects: LoginPage, DashboardPage, InventoryPage
components/      Shared UI parts: Toast, Spinner
apis/            API clients: AuthApi, InventoryApi
fixtures/        Give each test its page objects and logged-in API clients, and clean up after it
data/            Test data (JSON) and a factory that makes names and SKUs unique
config/          Base URL and users; credentials can come from environment variables
```

- **Pages** expose business actions such as `addProduct()`. Assertions stay in the tests.
- **Components** are written once and reused, for example the toast used by every module.
- **Fixtures** handle setup and cleanup, so tests only describe behaviour.
- **Unique test data** (timestamp + random suffix) lets tests run in parallel without collisions.
- **Locators:** `data-testid`, role and label first. XPath only as a last resort.

### 1.3 Implementation

Flow: log in, open Inventory, add a product, click Save, verify the success toast.

```
// tests/ui/inventory.spec.js
test('Store Admin can add a product @smoke', async ({ loginPage }) => {
  const { name, price } = products.addProduct;   // from data/products.json
  const productName = uniqueProductName(name);   // unique per run

  const dashboard = await loginPage.loginAs(users.admin.email, users.admin.password);
  const inventory = await dashboard.openInventory();
  await inventory.addProduct(productName, price);

  // Web-first assertions retry automatically until timeout
  await expect(inventory.toast.root).toBeVisible();
  await expect(inventory.toast.root).toContainText(products.successToastText);
});
```

```
// pages/inventory.page.js
async addProduct(name, price) {
  await this.page.getByLabel('Product Name').fill(name);
  await this.page.getByLabel('Price').fill(price);

  // Start waiting for the API call before clicking Save
  const saveResponse = this.page.waitForResponse(
    (res) => res.url().includes('/api/v1/inventory/items') && res.request().method() === 'POST'
  );
  await this.page.getByRole('button', { name: 'Save' }).click();
  expect((await saveResponse).status()).toBe(201);
}
```

```
// components/toast.js
class Toast {
  constructor(page) {
    this.root = page.getByRole('alert');
  }
}
```

The repository also includes `LoginPage`, `DashboardPage` and two more UI tests: the new product appears in the table, and a product from Tenant A is not visible to Tenant B.

### 1.4 Stability Without Hardcoded Sleeps

Every wait is tied to something that really happens in the app, never to a fixed time.

- **Web-first assertions** such as `await expect(toast).toBeVisible()` retry until they pass or time out.

- **Loading spinners:** `await expect(spinner).toBeHidden()` before using the content behind it.
- **Network delays:** `page.waitForResponse()` waits for the API call behind the UI change, as in the Save step above.
- **Async tables:** assert on the data itself, for example `await expect(row).toHaveCount(1)`, not just on the table being present.
- **Stable locators and unique data,** so tests do not depend on each other or on run order.
- **Retries only in CI,** with a trace recorded on the first retry. A flaky test is investigated and fixed, not hidden.

## Part 2: API Testing Strategy

Endpoint: `POST /api/v1/inventory/items` with `Authorization: Bearer <token>` and payload `{ "item_name": "Wireless Mouse", "sku": "MS-001", "quantity": 50, "price": 25.00, "category_id": 3 }`.

### 2.1 Test Scenarios

| ID | Type | Input | Expected |
|---|---|---|---|
| P1 | Positive | Valid token and valid payload | **201 Created.** The body returns the sent fields plus a new `id`. A GET by `id` returns the same item. |
| P2 | Positive | Boundary values: `quantity: 0`, `price: 0.01`, `item_name` of maximum length (100) | **201 Created.** Values are stored exactly as sent. |
| N1 | Negative | Missing, invalid or non-Bearer token | **401 Unauthorized.** No item is created. |
| N2 | Negative | SKU that already exists in the same tenant | **409 Conflict.** No second item is created. |
| N3 | Negative | Invalid values: `price: -5`, `quantity: "fifty"`, `category_id: 9999` | **400 / 422** with an error for the invalid field. No item is created. |

**Multi-tenant and permission risks** (also automated):

- A Cashier token (not an admin) gets **403 Forbidden**.
- A `category_id` that belongs to another tenant is rejected.
- The same SKU in a different tenant is allowed, because SKUs are unique per tenant.
- Tenant B gets **404** when reading an item created by Tenant A.

The exact validation status (400 or 422) should be confirmed against the API contract.

### 2.2 Postman Automation

**1. Dynamic token (collection-level Pre-request Script).** Before any request runs, the script logs in, saves the token in an environment variable, and fetches a new one only when the current token has expired.

```
const token = pm.environment.get("access_token");
const expiry = Number(pm.environment.get("token_expiry") || 0);

if (!token || Date.now() >= expiry) {
  pm.sendRequest({
    url: pm.environment.get("base_url") + "/api/v1/auth/login",
    method: "POST",
    header: { "Content-Type": "application/json" },
    body: {
      mode: "raw",
      raw: JSON.stringify({
        email: pm.environment.get("admin_email"),
        password: pm.environment.get("admin_password")
      })
    }
  }, (err, res) => {
    const { access_token, expires_in } = res.json();
    pm.environment.set("access_token", access_token);
    pm.environment.set("token_expiry", Date.now() + (expires_in - 60) * 1000);
  });
}
```

- In the collection's **Authorization** tab: type **Bearer Token**, value `{{access_token}}`. Every request uses it automatically.
- The email and password are stored as secret environment variables, not written in the collection.

**2. Unique SKU (request Pre-request Script),** so the request can run many times without a 409:

```
pm.variables.set("sku", "MS-" + Date.now());
```

Request body: `"sku": "{{sku}}"`.

**3. Tests tab assertions:**

```
pm.test("Status is 201 Created", () => {
  pm.response.to.have.status(201);
});

pm.test("Response returns the same SKU", () => {
  const body = pm.response.json();
  pm.expect(body.sku).to.eql(pm.variables.get("sku"));
});
```

The collection can also run from the command line with **Newman** (`newman run collection.json -e environment.json`), for example in a CI pipeline.

# Part 3: CI/CD & Database Validation

## 3.1 Database Testing

```
SELECT
    p.id,
    p.item_name,
    p.sku,
    p.quantity,
    p.price,
    c.name AS category_name
FROM products p
LEFT JOIN categories c ON c.id = p.category_id
WHERE p.sku = 'MS-001';
```

**Expected result:** exactly **one row**, with `category_name = 'Electronics'`, and `item_name`, `quantity` and `price` matching the API request.

**Why LEFT JOIN:** if the item exists but its category is wrong or missing, the row still appears, with an empty or different `category_name`. So the result shows clearly whether the item is missing or was assigned to the wrong category.

In a multi-tenant database, add `AND p.tenant_id = <test tenant>` so the query only checks the test tenant's data.

## 3.2 CI/CD Integration

The tests run in **GitHub Actions** (`.github/workflows/electropi-ci.yml`) on every pull request, every push to `main`, every night, and on demand.

1. **API job:** installs dependencies, starts the application, and runs the API tests.
2. **UI job:** starts **only if the API job passes** (`needs: api-tests`). It runs the UI tests split across 2 parallel machines (sharding).
3. **Reports:** each job uploads the HTML report, plus screenshots, videos and traces of failed tests, even when tests fail.
4. **Secrets:** credentials come from GitHub secrets, never from the code.

Simplified version of the workflow:

```yaml
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright test tests/api

  ui-tests:
    needs: api-tests            # UI tests run only if the API tests pass
    runs-on: ubuntu-latest
    strategy:
      matrix:
        shard: [1, 2]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test tests/ui --shard=${{ matrix.shard }}/2
```

In this repository the "test environment" is the mock app. For a real project, a deploy step to a QA environment runs first, and the tests point at it with `ELECTROPI_BASE_URL`.

### 3.3 Execution Strategy

API tests are fast and stable, so they run on every change. UI tests are slower, so only the critical flows run early and the full suite runs later.

| Stage | API tests | UI tests |
|---|---|---|
| Pull request | Smoke tests on key endpoints | Smoke only (for example: add product) |
| Merge to `main` / deploy to QA | Full API suite, including negative and tenant tests | UI smoke tests |
| Before release | Full suite | Full UI suite, cross-browser |
| Nightly | Full suite | Full UI suite, cross-browser |

**Why:** API tests find most business-logic bugs within minutes, so developers get fast feedback. If the API is broken, the UI tests would fail anyway, so they are skipped to save time.